National University Of Technology Islamabad

Department Of Computer Science

Information Security Lab

Name	**MOHSINA RUBAAB**
Roll No	**F24609012**
Batch	**24**
Section	**CY**
Lab Number	**OEL 1**
Instructor Name	**Maam Zainab**

Task 1

Compile the given C program to use SHA1 function to compute hash of a message.

Question 1

In Task 1, you have computed hash for the message “Hello, OpenSSL!”. If you change the message slightly (e.g., remove exclamation mark ‘!’ or comma ‘,’ between words, change a capital letter to small letter, etc.), does the hash stay the same or does it change completely? Show your answer as an output on the terminal for three slightly different messages and state the reason.

```
ubuntu@ubuntu:~$ gcc -o example example.c -lssl -lcrypto
ubuntu@ubuntu:~$ ./example
Message: Hello, OpenSSL!
SHA1 hash of the message:8dddc632114e6aadd6bed490fda93069bbf92cd9
ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ gcc -o example example.c -lssl -lcrypto
ubuntu@ubuntu:~$ ./example
Message: hello, OpenSSL!
SHA1 hash of the message:7f38b56aa5290749b5007a2337fb2cb4004400f3
ubuntu@ubuntu:~$ S
```

```
ubuntu@ubuntu:~$ gcc -o example example.c -lssl -lcrypto
ubuntu@ubuntu:~$ ./example
Message: hello OpenSSL!
SHA1 hash of the message:50be628e3ae2c88b5c9f4452de90e2e53f6bcd75
```

Message Variant	SHA-1 Hash Output
Hello, OpenSSL!	8dddc632114e6aadd6bed490fda93069bbf92cd9
Hello, OpenSSL	7f38b56aa5290749b5007a2337fb2cb4004400f3
hello OpenSSL!	50be628e3ae2c88b5c9f4452de90e2e53f6bcd75

Explanation

- When you changed the message slightly by removing the comma, or by changing the capital H to lowercase the SHA-1 hash changed completely.
- This shows the **avalanche effect**:
  In cryptographic hash functions, even a one-bit change in the input produces a completely different output.
- The SHA-1 algorithm always produces a **160-bit (20-byte)** hash, but the *values* differ drastically for small input differences.

Question 2

Does the length of the input message affect the length of the SHA-1 hash? Try computing the hash for different lengths of input (short and long messages) and compare the hash lengths.

```
ubuntu@ubuntu:~$ ./example
Message: Hello, OpenSSL!
SHA1 hash of the message:8dddc632114e6aadd6bed490fda93069bbf92cd9
Hash Length: 20 bytes (40 hex Characters)

ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ ./example
Message: hello, OpenSSL!
SHA1 hash of the message:7f38b56aa5290749b5007a2337fb2cb4004400f3
Hash Length: 20 bytes (40 hex Characters)

ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ ./example
Message: hello OpenSSL!
SHA1 hash of the message:50be628e3ae2c88b5c9f4452de90e2e53f6bcd75
Hash Length: 20 bytes (40 hex Characters)

ubuntu@ubuntu:~$
```

Message	Length of Input	SHA-1 Hash Output	Hash Length
Hello, OpenSSL	13 characters	8dddc632114e6aadd6bed490fda93069bbf92cd9	20 bytes (40 hex characters)
hello, OpenSSL	13 characters	7f38b56aa5290749b5007a2337fb2cb4004400f3	20 bytes (40 hex characters)
hello OpenSSL	12 characters	50be628e3ae2c88b5c9f4452de90e2e53f6bcd75	20 bytes (40 hex characters)

Explanation

- The SHA-1 algorithm always generates a fixed-size output of 160 bits = 20 bytes = 40 hexadecimal characters, no matter how long or short the input is.
- The *content* and *length* of the message only affect the hash value, not the hash size.
- SHA-1 processes data internally in 512-bit blocks and compresses it into a single 160-bit digest.

Question 3

Modify the code to experiment with different SHA functions (SHA-1, SHA-224, SHA-256, SHA-384, SHA-512).

- **Update your program to allow switching between different SHA functions (SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512).**
- **For each hash function, compute the hash of the same message and compare the output length and security level.**
- **Discuss the differences in hash length and security between the various SHA algorithms.**

```
ubuntu@ubuntu:~/Desktop$ nano sha_compare.c
ubuntu@ubuntu:~/Desktop$ gcc -o sha_compare sha_compare.c -lssl -lcrypto
ubuntu@ubuntu:~/Desktop$ ./sha_compare
Original Message: Hello, OpenSSL!

SHA-1 Hash: 8dddc632114e6aadd6bed490fda93069bbf92cd9 (Length: 20 bytes)
SHA-224 Hash: 66dffde342166fe01942f500ddfabb767093deaf76a422b227a62138 (Length: 28 bytes)
SHA-256 Hash: 63ee7f365450f9586dbb31c9b59db63817797e04ea1014dd5a8ac6615d44fac1 (Length: 32 bytes)
SHA-384 Hash: bb7f25215172dc09f6b412c6a3f7c8b80bd52899743a28540bd075c698b5a74695a3256a9ced157124851e1e232dd720 (Length:
48 bytes)
SHA-512 Hash: 279903aed7cb1f403354c78c47f9bd6ce28bd43c15a8bb2960b5f32f652ae50d1c5446b0bc41107fec1f070895dc762f5c03b0b647
495c87640923fbb6300cec (Length: 64 bytes)
ubuntu@ubuntu:~/Desktop$
```

Algorithm	Hash Output (Truncated)	Length (Bytes)
SHA-1	8dddc632114e6aadd6bed490fda93069bbf92cd9	20
SHA-224	66dffde342166fe01942f500ddfabb767093deaf76a422b227a62138	28
SHA-256	63ee7f365450f9586dbb31c9b59db63817797e04ea1014dd5a8ac6615d44fac1	32
SHA-384	bb7f25215172dc09f6b412c6a3f7c8b80bd52899743a28540bd075c698b5a7469b...	48
SHA-512	279903aed7cb1f403354c78c47f9bd6ce28bd43c15a8bb2960b5f32f652ae50d...	64

Explanation

Each SHA algorithm produces a unique hash value for the same input message but with a different fixed output length.

- The output size (hash length) does not depend on how long or short the message is.
- The higher the bit length of the algorithm, the larger and more secure the hash output becomes.

Algorithm	Hash Size	Relative Security	Status / Usage
SHA-1	160-bit	Weak	Deprecated – collisions found
SHA-224	224-bit	Moderate	Used for shorter outputs
SHA-256	256-bit	Strong	Widely used (TLS, Blockchain)
SHA-384	384-bit	Very Strong	Used in high-security systems
SHA-512	512-bit	Extremely Strong	Used in critical security systems

Security Analysis

- **Collision Resistance:**
  Bigger hash sizes make it much harder for two different messages to have the same hash.
  For example, SHA-512 is stronger and safer than SHA-1.
- **Preimage Resistance:**
  It is almost impossible to find the original message from its hash.
  Longer hashes make this even more difficult.
- **Practical Security:**
  SHA-1 is no longer safe because real attacks have broken it.
  SHA-256, SHA-384, and SHA-512 are still secure and used in modern systems.

Question 4

Change one character in the original message and encrypt it again using AES. How different is the ciphertext? Explain why this happens in AES encryption.

```
ubuntu@ubuntu:~/Desktop$ ./aes_example_modified
Original Message: Hello, OpenSSL!
Original Ciphertext: 4b4a69cf5ff3f6fbe0b3c1c31354c922
Decrypted Original Message: Hello, OpenSSL!

Modified Message: Hello OpenSSL!
Modified Ciphertext: 2c4214bb162c79e3ff32bda2913bcefe
Decrypted Modified Message: Hello OpenSSL!
ubuntu@ubuntu:~/Desktop$
```

Type	Message	Ciphertext (Hexadecimal)
Original Message	Hello, OpenSSL!	4b4a69cf5ff3f6fbe0b3c1c31354c922
Modified Message	Hello OpenSSL!	2c4214bb162c79e3ff32bda2913bcefe

Both messages differ only by one character the comma was removed after Hello Despite this small change, the ciphertext changed completely.

Reason:
AES encryption uses complex mathematical operations on blocks of data (substitution, permutation, and mixing).
Because of the **avalanche effect**, even a one-bit or one-character change in the plaintext causes large, unpredictable changes in the ciphertext.

This ensures that AES provides strong security — no pattern or similarity can be seen between the ciphertexts of similar plaintexts.

Question 5

Does cipher text length change with a change in the length of plaintext when using AES encryption? Explain with the help of screenshots.

```
ubuntu@ubuntu:~/Desktop$ ./aes_example_modified
Short Message: Hi
Ciphertext for Short Message: edbc6c030ed4c434744d8c956d78e0b1
Ciphertext Length for Short Message: 16 bytes

Long Message: This is a longer message to test.
Ciphertext for Long Message: e108a3d780e3044dec76151c87ff612743d9105fd8724ad6e00c6048bf978a968237fe09b76e834b2d60f54a678
18d3d
Ciphertext Length for Long Message: 48 bytes
ubuntu@ubuntu:~/Desktop$
```

Plaintext Message	**Plaintext Length**	**Ciphertext Length**	**Explanation**
"Hi"	2 bytes	16 bytes	Padded to one AES block
"This is a longer message to test."	~31 bytes	48 bytes	Encrypted in 3 AES blocks (3 × 16 = 48 bytes)

Explanation

The ciphertext length changes as the plaintext length changes, but not proportionally.
This happens because AES is a block cipher that encrypts data in fixed-size blocks of 16 bytes (128 bits).

- If the plaintext is shorter than 16 bytes, AES adds padding to make it exactly one block (16 bytes).
- If the plaintext is longer, AES divides it into multiple 16-byte blocks and encrypts each block separately.